

Gyuricza Márk Attila - PROJEKTFELADAT

Téma: Hőmérséklet-riasztó rendszer Raspberry Pi és Node-RED alapokon

Készítette: Gyuricza Márk Attila 13. b

Tantárgy: Rendszerüzemeltetés és IoT

Dátum: 2025. 12. 10.

TARTALOMJEGYZÉK

1. Projekt célja	3
2. Felhasznált eszközök	3
3. A projekt megvalósítása	3
3.1. Operációs rendszer telepítése	4
3.2. Node-RED környezet előkészítése	4
3.3. Szenzor beállítása és azonosítása	5
3.4. Hardveres bekötés	6
3.5. Telegram Bot létrehozása és beállítása	7
4. Projekt felépítése (Flow)	10
5. Projekt működése	10
6. Tesztelés és Eredmények	11
7. Összegzés	13
8. Forrásjegyzék	14

1. Projekt célja

A projektem egy **hőmérséklet-riasztó rendszer** létrehozása volt, amely érzékeli a környezeti hőmérséklet változását és hatékony védelmet nyújthat olyan kritikus infrastruktúrák számára, mint például a **szerverszobák** vagy **hűtőházak**, ahol a hirtelen hőmérséklet-emelkedés súlyos károkat okozhat. Fő feladata, hogy túlmelegedés esetén azonnal egy figyelmeztető értesítést küldjön a felhasználó okostelefonjára a Telegram alkalmazáson keresztül, miközben grafikus felületen a dashboardon (SCADA felületen) is megjeleníti a szenzorral mért hőmérséklet adatokat.

2. Felhasznált eszközök

A feladat megvalósításához az alábbi eszközöket és szoftvereket használtam:

- **Hardver:** Raspberry Pi 5, 1-Wire hőmérsékletszenzor (DS18B20).
- **Szoftver:** Raspberry Pi Imager, Node-RED környezet
- **Kommunikáció:** Telegram BOT API (az értesítésekhez).



1. ábra Fizikai eszközök (Raspberry Pi 5 és kiegészítők)

3. A projekt megvalósítása

A rendszer megvalósítása a szoftveres környezet kialakításával és az operációs rendszer telepítésével kezdődött, majd ezt követte a hardver összeállítása és a fizikai bekötés.

3.1. Operációs rendszer telepítése

A rendszer alapját a Raspberry Pi hardver és a rajta futó Linux alapú operációs rendszer képezi. A telepítéshez a gyártó hivatalos eszközét használtam.

A **Raspberry Pi Imager** egy hivatalos, ingyenes szoftver, amely leegyszerűsíti a Raspberry Pi operációs rendszereinek telepítését SD-kártyára. Elsődleges feladata a rendszerkép (image) letöltése és felírása, valamint a szükséges konfigurációk előzetes elvégzése. Segítségével egy gombnyomással indítható SD-kártyát készíthetünk a miniszámítógéphez.

A beállítások során a hálózati azonosítót (hostname) **pi2**-re állítottam. Az SD-kártya behelyezése, valamint a tápellátás és a hálózati kábel csatlakoztatása után a Raspberry Pi azonnal elérhetővé vált a hálózaton, készen a Node-RED telepítésére.

3.2. Node-RED környezet előkészítése

A Node-RED telepítéséhez a számítógépen a **PuTTY** szoftver segítségével, SSH kapcsolaton keresztül csatlakoztam a Raspberry Pi-hez. Első lépésként telepítettem a `curl` csomagot (az `apt install curl` paranccsal), amely elengedhetetlen a telepítőfájlok letöltéséhez.

Ezt követően lefuttattam a hivatalos telepítő scriptet a következő paranccsal:

```
bash <(curl -sL https://github.com/node-red/linux  
installers/releases/latest/download/update-nodejs-and-nodered-deb)
```

A telepítés végeztével a `node-red` paranccsal indítottam el az alkalmazást. A Node-RED indítása után a böngészőben a **pi2:1880** címet megnyitva kezdtem el a flow létrehozását.

A Node-RED-en belül a "Palette" menüben telepítenem kellett a szükséges kiegészítőket:

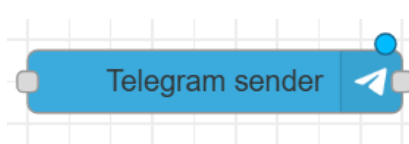
- `node-red-contrib-1wire`: a szenzorhoz.
- `node-red-dashboard`: a hőmérséklet diagramhoz.
- `node-red-contrib-telegrambot`: hogy az értesítést el tudja küldeni a Telegram alkalmazásba.



2. ábra
node-red-contrib-1wire



3. ábra
node-red-dashboard



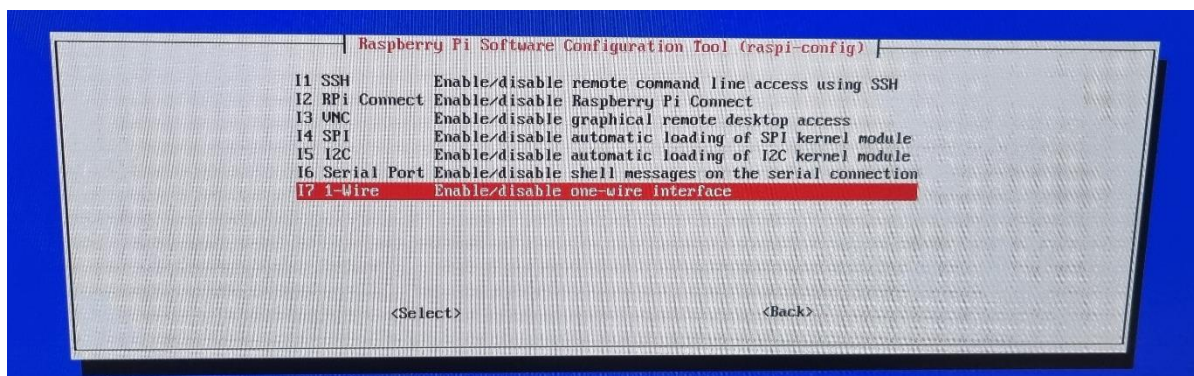
4. ábra
node-red-contrib-telegrambot

3.3. Szenzor beállítása és azonosítása

A hőmérséklet mérésére a **DS18B20** típusú digitális szenzort alkalmaztam, mivel az ipari környezetben is népszerű, megbízható mérési eredményeket ad. A csatlakoztatott DS18B20 hőmérőt a 1-Wire kommunikációs protokollt használja. A technológia nagy előnye, hogy az adatátvitelhez mindössze egyetlen vezetékre és a földelésre van szükség, ami leegyszerűsíti a bekötést és lehetővé teszi a pontos, digitális adatátvitelt egyetlen adatvezetéken keresztül, -55°C és $+125^{\circ}\text{C}$ közötti tartományban. A szenzor közvetlenül a Raspberry Pi GPIO portjára csatlakoztatható, ezért nincs szükség külön analóg-digitális átalakítóra. Ennek előnye, hogy minden eszköz egyedi gyári azonosítóval (szériaszámmal) rendelkezik, így a rendszer pontosan meg tudja különböztetni őket.

Az **1-Wire interfész engedélyezése** ahhoz, hogy a Raspberry Pi kommunikálni tudjon a digitális szenzorral, szoftveresen is engedélyezni kellett az 1-Wire protokollt. Ezt a **raspi-config** beállítóprogram segítségével végeztem el.

Az **Interface Options** majd a **1-Wire** menüpontban az "Enable" kiválasztásával utasítottam a rendszert, hogy induláskor automatikusan töltsse be a szükséges kernel modulokat (w1-gpio, w1-therm).



5. ábra Raspi-config beállítóprogram

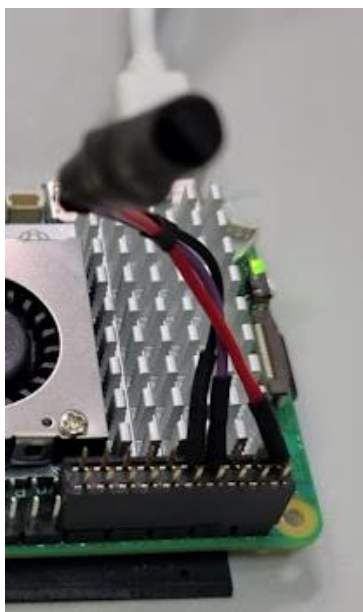
3.4. Hardveres bekötés

A DS18B20 hőmérséklet szenzort közvetlenül a Raspberry Pi GPIO (General Purpose Input/Output) tűskéire csatlakoztattam. Mivel ez egy digitális eszköz, három vezetékre volt szükség a működéséhez, tápellátásra, földelésre és az adatátvitelre.

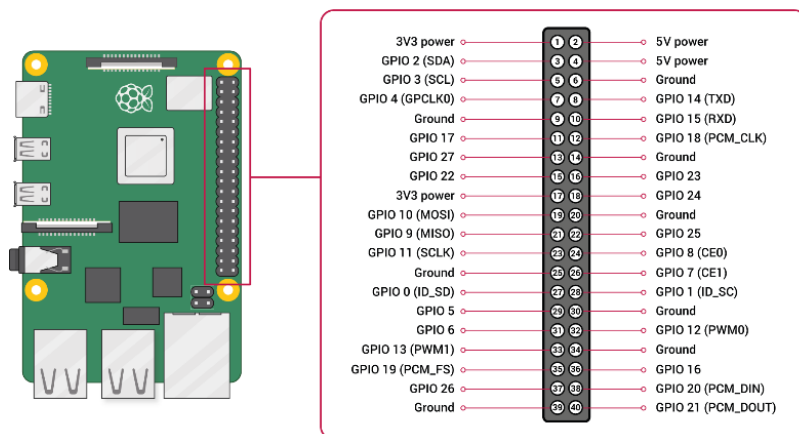
A bekötés részletezése:

- **Tápellátás (Piros vezeték - Pin 1):** A bal felső sarokban található legelső tűske, amely a **3.3V Power** kimenet. A DS18B20 szenzornak stabil 3.3V feszültségre van szüksége. Mivel ez a tűske közvetlenül a tápegységről kapja az áramot (nem szoftveresen kapcsolható), ideális a szenzor folyamatos működtetéséhez.
- **Földelés (Fekete vezeték - Pin 6):** A jobb oldali oszlop harmadik tűskéje, a **GND (Ground)**. Az áramkör zárásához szükség van a negatív pólusra, földelésre. Bár a Raspberry Pi-n több GND tűske is van (pl. Pin 9, Pin 14), fizikailag a 6-os tűske helyezkedik el a legközelebb a másik két vezetékhez, így a kábelezés a lehető legrendezettebb maradt.
- **Adatkommunikáció (Lila vezeték - Pin 7):** A bal oldali oszlop negyedik tűskéje. Fizikailag a 7-es, de a szoftver számára ez a **GPIO 4-es** port. Ez a legfontosabb választás. A Raspberry Pi operációs rendszere gyárilag úgy van konfigurálva, hogy az 1-Wire eszközöket (mint a hőmérő) alapértelmezetten a GPIO 4-es porton keresi. Ennek használatával elkerülhető volt a bonyolult rendszerfájlok módosítása, és a rendszer automatikusan felismerte a szenzort.

A csatlakozásoknál a biztonság érdekében zsugorcsöves szigetelést használtam.



6. ábra Hardveres bekötés



7. ábra Raspberry Pi GPIO portjainak lábkiosztása (Pinout diagram)

A szenzor egyedi hardveres azonosítójának megszerzéséhez a Raspberry Pi termináljában a következő parancsot futtattam:

```
ls /sys/bus/w1/devices/
```

A kapott listából a **28**-cal kezdődő mappa neve volt a szenzor azonosítója: **28-0b24822c7a40**. Ezt az azonosítót a Node-RED 1-Wire node-jának beállításainál adtam meg, így a szoftveres is hozzárendelésre került az egyetlen csatlakoztatott hőmérséklet-szenzorhoz, biztosítva a pontos adatkiolvasást.

3.5. Telegram Bot létrehozása és beállítása

Mi a Telegram? A Telegram egy felhőalapú, azonnali üzenetküldő szolgáltatás, amely nagy hangsúlyt fektet a sebességre és a biztonságra. Az ipari informatikában és az IoT fejlesztésekben azért népszerű, mert **nyílt API-val** (alkalmazásprogramozási felülettel) rendelkezik. Ez lehetővé teszi, hogy külső rendszerek (például a Node-RED) automatizált üzeneteket és riasztásokat küldjenek rajta keresztül egy bot segítségével, anélkül, hogy ehhez emberi beavatkozásra lenne szükség.

A Bot fogalma: A bot (a robot szó rövidítése) egy olyan automatizált szoftveres alkalmazás, amely emberi beavatkozás nélkül, előre meghatározott algoritmusok alapján hajt végre feladatokat az interneten keresztül. A Telegram bot központi szerepet tölt be a rendszerben,

mint kommunikációs híd a Raspberry Pi és az okostelefon között, fogadja a Node-RED által küldött adatokat, majd azokat azonnali üzenet formájában továbbítja a felhasználónak.

A BotFather szerepe: A bot regisztrációját és alapbeállításait a **BotFather** segítségével végeztem el. Ez a Telegram saját, hivatalos rendszermodulja, gyakorlatilag egy speciális belső felhasználó, amely a botok létrehozásáért és adminisztrációjáért felel. Nem különálló szoftver, hanem a Telegram alkalmazáson belül érhető el.

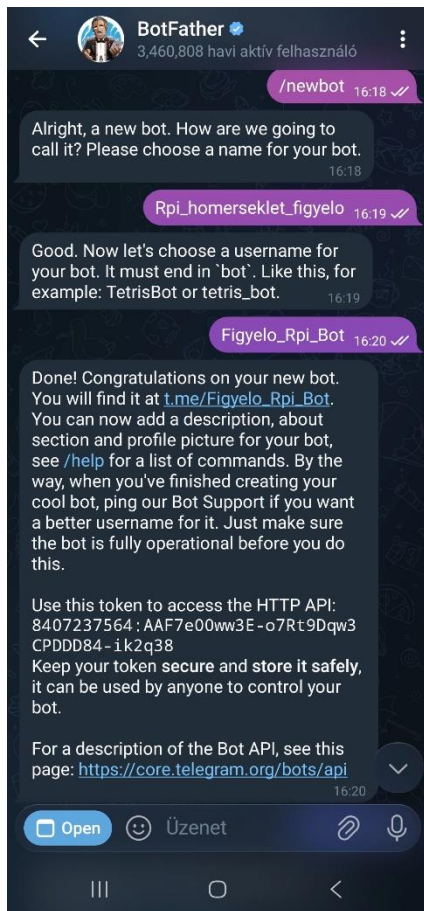
API Token: A BotFatheren keresztül generáltam a rendszer legfontosabb elemét, az API Tokent. Ez egy titkos, egyedi hozzáférési kulcs, amely lehetővé teszi a Node-RED szoftver számára, hogy azonosítsa magát a Telegram szerverein, és jogosultságot kapjon az üzenetküldésre.

Mivel az értesítéseket a saját okostelefonomra szerettem volna megkapni, szükségem volt a Telegram alkalmazásra és egy saját "robotra" (Bot), ami az üzeneteket küldi.

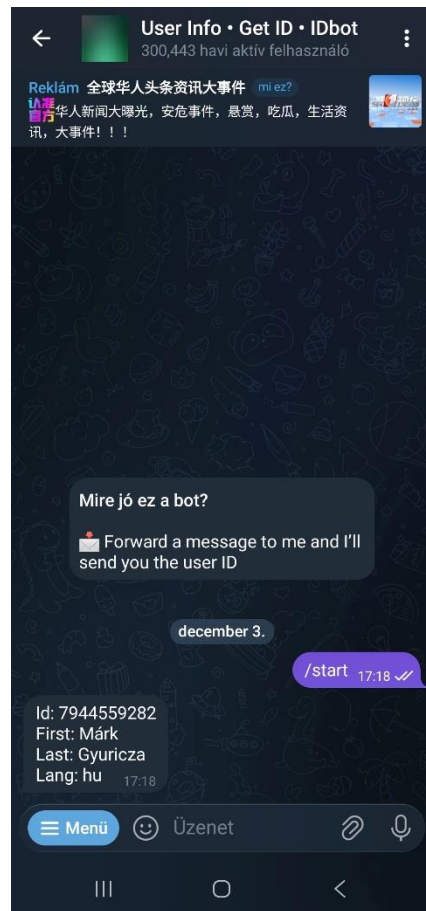
1. Bot regisztrálása (Token megszerzése): Letöltöttem a Telegram alkalmazást és regisztráltam. Ezután megkerestem a **BotFather** nevű hivatalos felhasználót. A `/newbot` parancs kiadásával létrehoztam az új botot, amihez a rendszer generált egy titkos hozzáférési kulcsot, az úgynevezett **API Token**.

2. Saját Chat ID megszerzése: A Token önmagában nem elég, a botnak tudnia kell a pontos címet (személyes azonosítót) is, ahová az üzenetet kézbesítenie kell. Ez a **Chat ID**. Ennek megszerzéséhez elindítottam egy beszélgetést az újonnan létrehozott botommal (rányomtam a *Start* gombra), majd egy segédprogrammal, User Info ID bot segítségével kiolvastam a saját, egyedi számsoromat.

3. Konfigurálás a Node-RED-ben: A Node-RED felületén a **Telegram Sender** node beállításában rögzítettem a kapott **Token**. A megszerzett **Chat ID**-t pedig a Function node kódjába vagy közvetlenül a Sender node mezőjébe illesztettem be. Így biztosítottam, hogy a riasztás kizárólag az én telefonomra érkezzen meg.

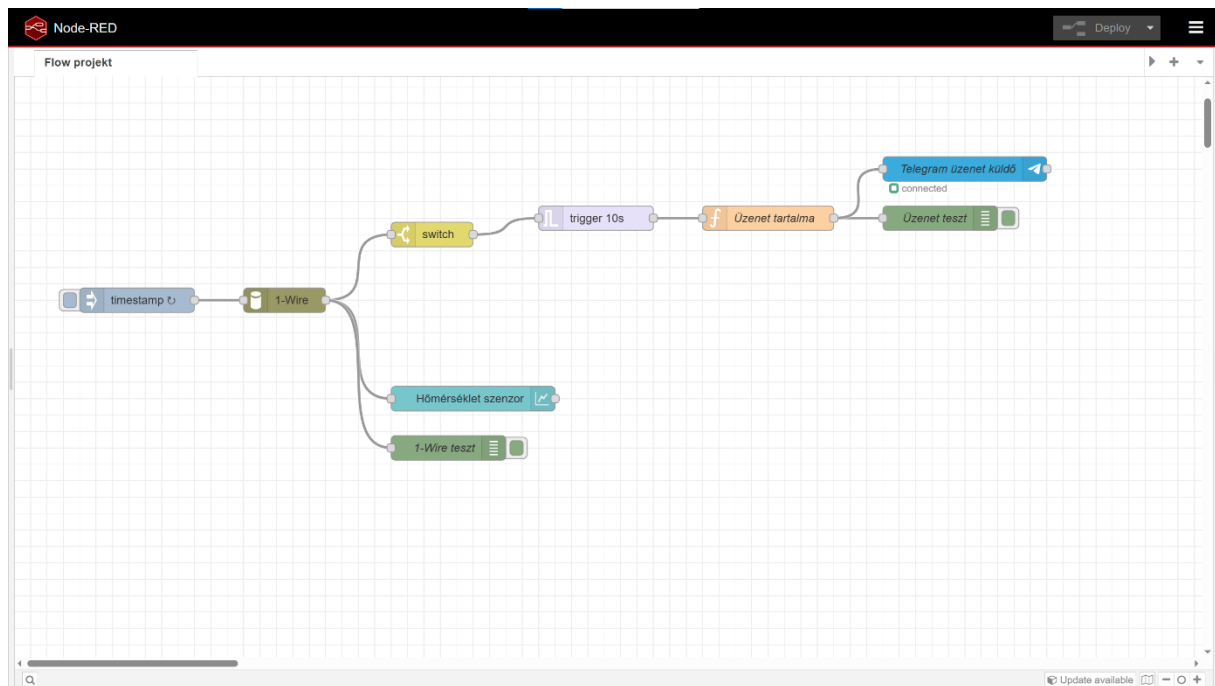


8. ábra A Bot létrehozása a BotFather segítségével



9. ábra Chat ID számsor megszerzése

4. Projekt felépítése (Flow)



10. ábra A Node-RED flow felépítése

A rendszer szoftveres megvalósítása a Node-RED vizuális fejlesztői környezetben történt. Az adatfolyam (Flow) balról jobbra halad, és az alábbi főbb csomópontokból (node-okból) épül fel:

- **Timestamp:** Ez az időzítő. Ez adja a jelet a rendszernek, hogy milyen gyakran (Nálam 5 másodpercenként) mérje meg a hőmérsékletet.
- **1-Wire:** A hardveres illesztésért felelős elem, amely kiolvassa a fizikai szenzor adatait.
- **Elágazás:** A mért adat két irányba halad tovább:
 1. **Megjelenítés:** A Dashboard felületre, ahol egy grafikon (Hőmérséklet szenzor node) mutatja az adatokat.
 2. **Riasztás:** A logikai ágra (switch, trigger, function), ami a figyelmeztetést kezeli.

5. Projekt működése

A riasztórendszer logikája a következőképpen működik:

1. **Szűrés (Switch node):** A beérkező hőmérsékletet vizsgálja. Csak akkor engedi tovább a jelet, ha az érték magasabb az adott °C-nál (30°C-ot állítottam be).

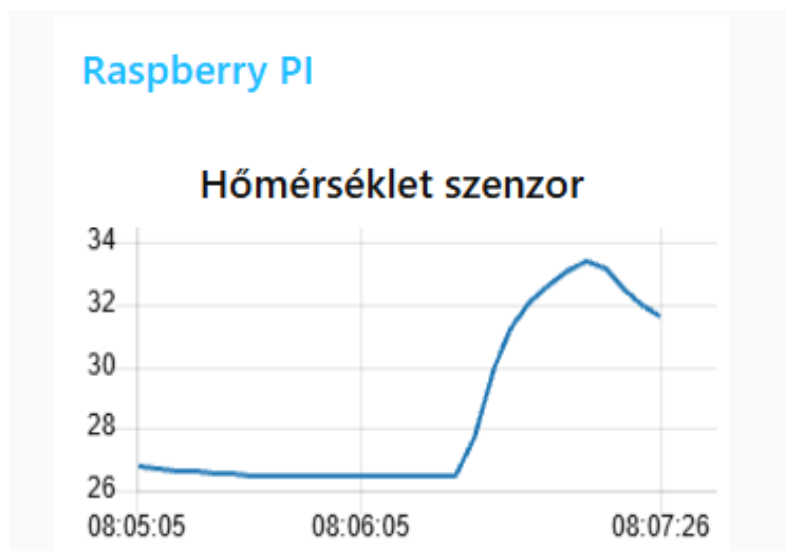
2. **Időzítés és védelem (Trigger node):** Ha a hőmérséklet magas, a jel a Trigger node-ba kerül. Ez egy adott várakozási időt iktat be. Feladata, hogy szabályozza az értesítések gyakoriságát, vagyis megakadályozza a "spam" üzeneteket, hogy ne kapjunk rengeteg üzenetet egymás után folyamatosan, ha ingadozik a hőmérséklet.
3. **Üzenet tartalma (Function node):** Ez a rész felel az üzenet tartalmáért. Átalakítja a mért adatot szöveggé, hogy hogyan kapja meg a felhasználó (pl. "Vigyázat! Túl meleg van! Jelenleg: 31 °C ").
4. **Üzenet küldése (Telegram sender):** Ez továbbítja az üzenetet az okostelefonra a Telegram alkalmazásba.

6. Tesztelés és Eredmények

A tesztelés során a rendszer megfelelően működött. Amikor a hőmérséklet emelkedett, a dashboardon a grafikon követte a változást, és a telefonomra is megérkezett a riasztás.

A tesztelés menete és tapasztalatok:

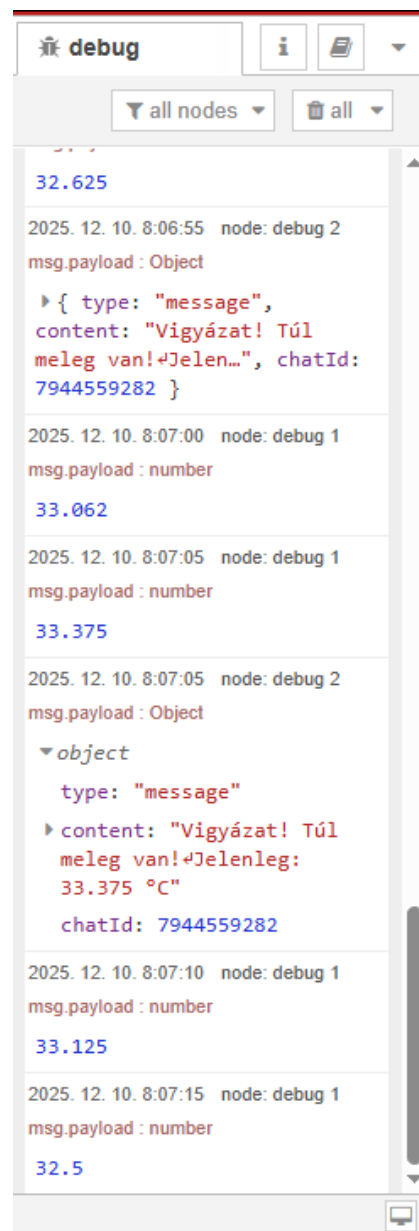
1. **Adatvizualizáció ellenőrzése:** A rendszer elindítása után megnyitottam a Node-RED Dashboard felületet a böngészőben. A teszt során a hőmérsékletszenzort mesterségesen melegítettem, kézmeleggel. A grafikonon azonnal, **valós időben** láthatóvá vált a hőmérséklet emelkedése, ami igazolja, hogy a 1-Wire kommunikáció és a Dashboard frissítése megfelelően működik.
2. **Riasztási logika vizsgálata:** Amikor a mért hőmérséklet átlépte a beállított biztonsági küszöbértéket (30°C-ot), a rendszer automatikusan működésbe lépett. A Node-RED *Debug* ablakában megjelent a generált figyelmeztető üzenet, ami igazolja a Switch és Function node-ok helyes beállítását.
3. **Értesítés megérkezése:** A folyamat végén ellenőriztem az okostelefonomat. A riasztási folyamat végén a rendszer sikeresen kézbesítette a valós idejű értesítést a Telegram végpontra. Az üzenet tartalmazta a figyelmeztető szöveget és az aktuális hőmérsékleti értéket is, így távolról is pontos információt kaptam a kritikus állapotról.



11. ábra A hőmérséklet emelkedése a Dashboard grafikonján



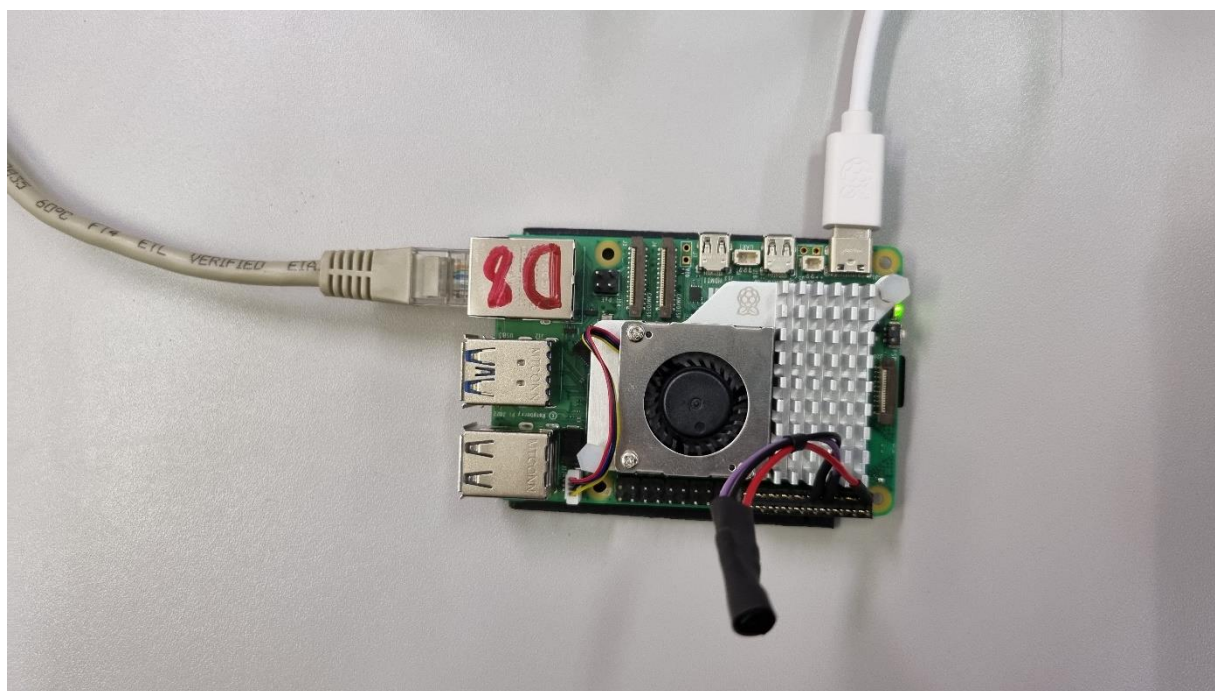
13. ábra A beérkezett riasztások a Telegram alkalmazásban a mért adatokkal



12. ábra A rendszer belső visszajelzése a határérték átlépésekor

7. Összegzés

Végeredményül a projektfeladat elkészítése során sikeresen összeállítottam egy működő **hőmérséklet-riasztó rendszert**, ami riasztást küld. A projekt elérte a célját a Raspberry Pi és a Node-RED segítségével a rendszer képes önállóan mérni a hőmérsékletet, és ha túlmelegedést észlel, azonnal értesítést küld a telefonomra.



14. ábra Az összeállított, működő rendszer

8. Forrásjegyzék

- Raspberry Pi Documentation: Hivatalos hardver-specifikációk és GPIO lábkiosztás

Forrás: <https://www.raspberrypi.com/documentation/>

- Node-RED Official Guide: Csomópontok (nodes) konfigurálása és flow-k kialakítása.

Forrás: <https://nodered.org/docs/>

- OpenJS Foundation: Node-RED Dashboard csomópontok és HMI vizualizációs útmutató.

Forrás: <https://flows.nodered.org/node/node-red-dashboard>

- Telegram Bot API: Bot létrehozása, API Token és Chat ID kezelése.

Forrás: <https://core.telegram.org/bots/api>

- DS18B20 Datasheet: Az 1-Wire digitális hőmérsékletszenzor technikai paraméterei és bekötési rajza.

Forrás: <https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf>